

Backend Storage & Query Migration to OpenSearch + S3

Technical Alignment Document for UI and Backend Teams

Date: March 19, 2026 (Updated)

Prepared by: OpenSearch UI Engineering Manager

Audience: Backend Engineering Team

Executive Summary

The backend team is implementing a **dual-track storage architecture**:

1. **Saved Objects API** → Rewritten to persist in **S3** (bypassing OpenSearch)
2. **Search/Query APIs** → Reimplemented on **OpenSearch** infrastructure

This document outlines critical alignment points, technical requirements, and execution strategy to ensure **zero frontend impact** during this migration.

Core Principle: Backend changes must be transparent to the UI layer. All existing OSD API contracts must remain stable.

Architecture Overview

Current vs. New Architecture



Migration Scope

API Category	# Endpoints	Storage Backend	Migration Approach	Impact
Saved Objects	17	S3	Rewrite with S3 SDK	HIGH - New codebase
Search/Query	12	OpenSearch	Migrate from legacy	MEDIUM - Backend swap
Console Proxy	3	OpenSearch	No change (passthrough)	LOW - Already OpenSearch
Data Importer	4	OpenSearch	Migrate from legacy	LOW
Other APIs	60+	Mixed/TBD	Case-by-case evaluation	VARIES

Key Change: Saved Objects no longer use OpenSearch .kibana index. All saved objects (dashboards, visualizations, index patterns, searches) will be stored as JSON files in S3.

1. Critical Alignment Points

1.1 API Contract Stability (Non-Negotiable)

Backend Team Must Guarantee:

Aspect	Requirement	Rationale
HTTP Endpoints	No changes to paths, methods, or URL parameters	Frontend has hardcoded endpoint references
Request Schema	Identical request body structure, query params, headers	Client-side validation and serialization depends on current schema
Response Schema	Same JSON structure, field names, data types	Frontend parsers and UI components expect exact field mappings
Error Responses	Maintain existing error codes and formats	Error handling logic relies on specific error structures
Authentication/Authorization	No changes to auth flow or headers	Session management and security interceptors are UI-owned

Verification Method: OpenAPI spec diff analysis (use openapi-spec.yaml as baseline)

Critical for S3-based Saved Objects:

- NDJSON import/export format must remain unchanged
- Object IDs and type conventions must be preserved
- Namespace isolation (multi-tenancy) must work identically
- Migration path from existing .kibana index to S3 must be transparent to users

1.2 Performance Characteristics by API Category

S3-Based Saved Objects API (17 endpoints):

- **Target p95 latency:** $\leq 200\text{ms}$ (current baseline $\sim 150\text{ms}$ for .kibana queries)
- **Critical operations:**
 - `/api/saved_objects/_find` - Must support pagination, filtering, sorting
 - `/api/saved_objects/{type}/{id}` - GET/PUT/DELETE must be $< 100\text{ms}$
 - `/api/saved_objects/_import` - Bulk import must handle 1000+ objects
 - `/api/saved_objects/_export` - Streaming export for large datasets
- **Concerns:**
 - S3 read consistency (eventual vs. strong consistency)
 - S3 list operations performance (for `_find` queries)
 - Caching strategy for frequently accessed objects

OpenSearch Search/Query API (12 endpoints):

- **Target p95 latency:** Within 10% of current baseline
- **Critical operations:**
 - `/internal/search/{strategy}` - Core search functionality
 - `/internal/_msearch` - Multi-search batching
 - `/api/console/proxy` - Direct OpenSearch proxy (no change needed)
- **Concerns:**
 - Connection pooling behavior
 - Query DSL compatibility
 - Aggregation performance

1.3 Functional Parity Requirements

Saved Objects API (S3-Based) - Zero Behavioral Changes:

1. CRUD Operations

- Create, Read, Update, Delete must work identically
- Bulk operations (bulk create, bulk update) must support same payload sizes
- Conflict resolution on duplicate IDs must behave the same

2. Search & Find

- `/api/saved_objects/_find` must support:
 - Type filtering (e.g., `type=dashboard`)
 - Full-text search across object attributes
 - Pagination (`page`, `perPage` params)
 - Sorting by multiple fields
 - Namespace filtering (for multi-tenancy)
- **Challenge:** S3 doesn't have native query capabilities like OpenSearch
- **Solution needed:** Metadata indexing layer (DynamoDB? Local cache?)

3. Import/Export

- NDJSON format must remain unchanged
- Import must handle conflict resolution (overwrite vs. skip)
- Export must support bulk download (potentially thousands of objects)
- **Challenge:** Large exports may need streaming from S3

4. Namespace Isolation (Multi-Tenancy)

- Workspace-level isolation must work identically
- Object sharing across namespaces must be preserved
- Default namespace behavior unchanged

5. Migrations

- Saved Objects versioning and migrations must still work
- Migration scripts must be S3-aware
- Rollback capability required

Search/Query API (OpenSearch-Based) - Maintain Parity:

1. Query Execution

- Query DSL passthrough for Console proxy
- Search strategies: default, async, DQL, PPL
- Aggregations and bucketing

2. Data Source Management

- Multi-cluster connectivity
 - Connection pooling behavior unchanged
-

2. Technical Proposal

2.1 Dual-Track Implementation Strategy

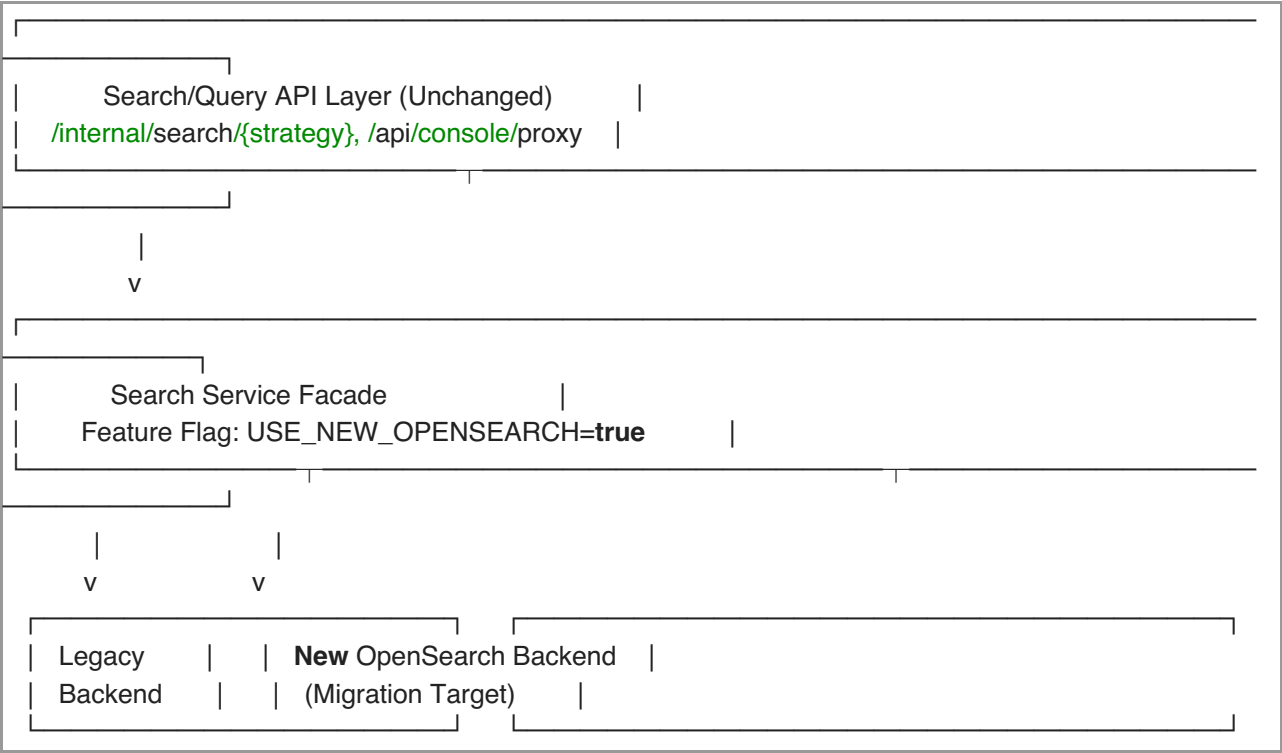
Track 1: Saved Objects → S3 (Higher Risk, New Architecture)



Key Design Questions for Backend Team:

- 1. Metadata Indexing:** How will `_find` queries work efficiently?
 - Option A: DynamoDB for metadata, S3 for object blobs
 - Option B: In-memory cache + full S3 scan (not scalable)
 - Option C: Local search index (SQLite/PostgreSQL) + S3 storage
 - Recommendation:** DynamoDB for queryable metadata + S3 for object storage
- 2. Caching Strategy:** How to minimize S3 read latency?
 - CDN caching (CloudFront) for frequently accessed objects
 - Application-level cache (Redis/Memcached)
 - Cache invalidation strategy on updates
- 3. Consistency Model:** S3 eventual consistency implications?
 - S3 now provides strong read-after-write consistency (as of Dec 2020)
 - Need to verify this works for multi-region deployments
- 4. Data Migration:** How to migrate existing `.kibana` index to S3?
 - One-time bulk migration script
 - Dual-write period (write to both OpenSearch and S3)
 - Migration verification (checksum validation)

Track 2: Search/Query → OpenSearch (Lower Risk, Backend Swap)



2.2 Implementation Phases

Phase 1: Foundation & Design (Weeks 1-3)

Track 1 (Saved Objects → S3):

- Finalize S3 storage architecture (metadata indexing strategy)
- Design S3 object key structure (e.g., {namespace}/{type}/{id}.json)
- Implement S3 repository with full CRUD operations
- Implement metadata indexing layer (DynamoDB/PostgreSQL)
- Build caching layer (Redis/CloudFront)
- Implement data migration script (OpenSearch .kibana → S3)

Track 2 (Search/Query → OpenSearch):

- Set up OpenSearch backend infrastructure
- Implement search service facade with feature flags
- Connection pooling and client configuration

Phase 2: Validation & Testing (Weeks 4-6)

Track 1 (Saved Objects → S3):

- Unit tests: All 17 Saved Objects endpoints
- Integration tests: Import/export workflows, bulk operations
- Performance testing: `_find` query latency, cache hit rates
- Data migration dry-run: Migrate staging .kibana to S3, validate checksums
- Shadow mode: Dual-write to both OpenSearch and S3, compare responses

Track 2 (Search/Query → OpenSearch):

- Integration tests: Search strategies, aggregations
- Performance benchmarking: Query latency comparison
- Shadow mode: Dual-query, response diff validation

Phase 3: Staged Rollout (Weeks 7-10)

Track 1 (Saved Objects → S3):

- Week 7: Internal dogfooding (10% traffic)
- Week 8: Beta customers (25% traffic)
- Week 9: 50% rollout
- Week 10: 100% rollout, deprecate OpenSearch .kibana index

Track 2 (Search/Query → OpenSearch):

- Week 7: 10% canary
- Week 8-9: 50% rollout
- Week 10: 100% rollout

Phase 4: Cleanup & Optimization (Weeks 11-12)

- Remove feature flags (optional)
- Remove legacy code paths
- Performance optimization based on production metrics
- Cost optimization (S3 storage tiers, caching tuning)

2.3 Technical Safeguards

Mandatory Before Production Rollout:

For S3-Based Saved Objects (Track 1):

1. API Contract Tests

- All 17 Saved Objects endpoints pass OpenAPI spec validation
- Response schema validation for every endpoint
- NDJSON import/export format unchanged

2. Data Integrity Tests

- Checksum validation: Migrated data matches source
- Zero data loss: All objects migrated successfully
- Namespace isolation verified (multi-tenancy)

3. Performance Benchmarks

- `_find` query latency $\leq 200\text{ms}$ (p95)
- GET single object $\leq 100\text{ms}$ (p95)
- Import/export throughput ≥ 100 objects/sec

4. Failure Mode Testing

- S3 unavailability: Graceful degradation with caching
- Metadata index failure: Fallback strategy defined
- Cache invalidation: Verify consistency

For OpenSearch Search/Query (Track 2):

1. Query Compatibility

- All Query DSL features supported
- Aggregation parity verified
- Scroll API behavior unchanged

2. Performance Benchmarks

- Search query latency within 10% of baseline (p95)
- Multi-search throughput unchanged

3. Execution Strategy

3.1 Team Responsibilities

Team	Track 1: Saved Objects → S3	Track 2: Search → OpenSearch
Backend	• S3 repository implementation • Metadata indexing (DynamoDB/PostgreSQL) • Caching layer (Redis/CloudFront) • Data migration tooling • Performance optimization	• OpenSearch backend setup • Search service facade • Query compatibility testing • Performance tuning
UI (Our Team)	• API contract validation • Integration test execution (Saved Objects) • UAT coordination • Import/export workflow validation • Go/no-go authority	• Search UI testing • Console proxy validation • Performance acceptance • Go/no-go authority
QA	• End-to-end testing • Performance validation • Data migration testing • Multi-tenancy testing	• Search strategy testing • Aggregation validation • Load testing
Infra/DevOps	• S3 bucket setup & permissions • DynamoDB provisioning • CloudFront CDN configuration • Monitoring & alerting	• OpenSearch cluster setup • Connection pooling config • Monitoring & alerting

3.2 Communication Cadence

- **Weekly Sync (All Teams):** Progress updates, blocker escalation, risk review
- **Bi-Weekly Architecture Review:** Deep-dive on S3 metadata indexing strategy, caching design
- **Pre-Rollout Review (48 hours before):** Go/no-go decision for each phase
- **Daily Standups (During Rollout - Weeks 7-10):** Quick sync, metrics review
- **Incident Response:** Backend + UI teams on-call during rollout windows

3.3 Success Criteria (Go/No-Go Gates)

Phase 2 → Phase 3 (Validation → Rollout):

Track 1 (Saved Objects → S3):

- All 17 API endpoints pass contract validation
- 100% Cypress test pass rate for Saved Objects workflows
- Data migration script validated (dry-run successful)
- Zero data loss in shadow mode (dual-write comparison)
- Performance: `_find` ≤ 200ms (p95), `GET` ≤ 100ms (p95)
- Metadata indexing layer proven to scale (1M+ objects)

Track 2 (Search → OpenSearch):

- All 12 search endpoints pass contract validation
- Query DSL compatibility verified
- Performance within 10% of baseline (p95)
- Zero errors in shadow mode

Phase 3 Milestones (Staged Rollout):

10% Canary:

- Error rate < 0.1%
- Latency within 10% baseline
- No data consistency issues reported

50% Rollout:

- Sustained performance for 48 hours
- No P0/P1 incidents
- User feedback positive (UAT group)

100% Rollout:

- Final sign-off from UI, QA, Backend, Infra leads
- Legacy backend (.kibana index) no longer in use
- Cost projections validated (S3 + DynamoDB costs acceptable)

3.4 Rollback Strategy

Immediate Rollback Triggers:

- Error rate increase > 1% absolute
- p95 latency increase > 25%
- Data loss or corruption detected
- S3/DynamoDB service outage with no failover

Rollback Procedures:

Track 1 (S3 Saved Objects):

- Feature flag flip: `USE_S3_BACKEND=false` → routes to legacy .kibana (30 seconds)
- Verify dual-write kept .kibana index up-to-date during shadow mode
- If dual-write missed updates: Emergency data sync from S3 back to OpenSearch

Track 2 (OpenSearch Search):

- Feature flag flip: `USE_NEW_OPENSEARCH=false` (30 seconds)
- Traffic routes back to legacy backend automatically

Post-Rollback:

- Post-mortem within 24 hours
 - Root cause analysis
 - Fix-forward plan or extended testing period
-

4. Risk Mitigation

4.1 Identified Risks

Risk	Impact	Mitigation
S3 metadata indexing insufficient	CRITICAL	Prototype and load-test DynamoDB/PostgreSQL solution early (Phase 1)
_find query performance unacceptable	HIGH	Implement aggressive caching, consider pre-computed indexes
Data migration errors	CRITICAL	Dry-run migrations, checksum validation, rollback scripts, dual-write period
S3 storage costs exceed budget	MEDIUM	Cost modeling in Phase 1, S3 lifecycle policies, compression
NDJSON import/export broken	HIGH	Dedicated integration tests for import/export workflows
Multi-tenancy isolation broken	HIGH	Security audit of S3 key structure, IAM policies, namespace filtering
API response schema drift	HIGH	OpenAPI spec validation in CI/CD pipeline
S3 eventual consistency issues	MEDIUM	Use S3 strong consistency (default), add retries for edge cases

4.2 Open Questions for Backend Team

S3 Architecture (Track 1):

1. **Metadata Indexing Strategy:** What's the technical design for _find queries?

- Recommended: DynamoDB (fast, scalable, AWS-native)
- Alternative: PostgreSQL RDS (more query flexibility)
- Need detailed design doc by end of Week 1

2. **Caching Strategy:** What's the cache invalidation approach?

- How to ensure cache consistency across multiple OSD instances?
- Cache TTL strategy?
- CDN vs. application-level caching?

3. **S3 Object Key Design:** What's the S3 key structure?

- Example: {tenant}/{namespace}/{type}/{id}.json?
- How to handle object versioning?
- Soft deletes vs. hard deletes?

4. **Data Migration Plan:** When and how to migrate existing .kibana data?

- Migration timeline?
- Downtime required (or zero-downtime via dual-write)?
- Verification process?
- Rollback plan if migration fails?

5. **Cost Projections:** What are the expected AWS costs?

- S3 storage cost per 1M objects?
- DynamoDB read/write capacity costs?
- CloudFront CDN costs?

- Comparison to current OpenSearch hosting costs?
6. **Multi-Region Strategy:** How to handle multi-region deployments?
- S3 cross-region replication?
 - DynamoDB global tables?
 - Latency implications?
-

5. UI Team Requirements from Backend

5.1 Deliverables Needed

Track 1 (S3 Saved Objects):

1. Architecture Design Doc

- S3 metadata indexing approach (DynamoDB/PostgreSQL)
- S3 object key structure
- Caching strategy
- Data migration plan
- **Due:** End of Week 1

2. Data Migration Plan

- Detailed steps, timeline, downtime estimate
- Rollback procedure
- Checksum validation approach
- **Due:** End of Week 2

3. Performance Test Results

- _find query latency benchmarks
- Import/export throughput
- Cache hit rate analysis
- **Due:** End of Phase 2 (Week 6)

4. Updated OpenAPI Spec

- Diff report showing zero breaking changes
- **Due:** 1 week before Phase 3 rollout

Track 2 (OpenSearch Search):

1. Performance Test Results

- Query latency comparison (legacy vs. new)
- **Due:** End of Phase 2 (Week 6)

2. Feature Flag Configuration

- Documentation on how to toggle backends
- **Due:** Phase 1 completion

5.2 Access Requests

- ☐ Read-only access to S3 buckets (for debugging)
 - ☐ Read-only access to DynamoDB tables (for debugging)
 - ☐ Read-only access to OpenSearch backend logs
 - ☐ Metrics dashboards for both S3 and OpenSearch backends
 - ☐ Canary deployment control (ability to adjust traffic percentage)
-

6. Next Steps

Immediate Actions (This Week):

1. **Backend Team:**

- ☐ Confirm alignment on dual-track strategy (S3 + OpenSearch)
- ☐ Provide S3 architecture design doc (metadata indexing, caching)
- ☐ Provide data migration plan for .kibana → S3
- ☐ Provide cost projections (S3 + DynamoDB vs. current)

2. **UI Team (Our Team):**

- ☐ Prepare OpenAPI spec baseline (openapi-spec.yaml)
- ☐ Identify critical Saved Objects workflows for UAT
- ☐ Prepare integration test suite for Saved Objects API

3. **Both Teams:**

- ☐ Review and approve this technical proposal
- ☐ Schedule weekly syncs starting next week
- ☐ Set up shared Slack channel: #backend-migration-2026

Weekly Milestones (First Month):

- **Week 1:** Architecture design finalized, S3 metadata strategy approved
 - **Week 2:** S3 repository implementation started, migration plan approved
 - **Week 3:** Feature flags implemented, shadow mode testing begins
 - **Week 4:** Integration testing for both tracks
-

Appendix A: API Scope Summary

Track 1: Saved Objects API → S3 (17 Endpoints)

Endpoint	Method	Description
/api/saved_objects/_find	GET	Search/filter saved objects
/api/saved_objects/{type}/{id}	GET	Get single object
/api/saved_objects/_bulk_get	POST	Bulk retrieve objects
/api/saved_objects/{type}/{id?}	POST	Create object
/api/saved_objects/{type}/{id}	PUT	Update object
/api/saved_objects/{type}/{id}	DELETE	Delete object
/api/saved_objects/_bulk_create	POST	Bulk create objects
/api/saved_objects/_bulk_update	PUT	Bulk update objects
/api/saved_objects/_import	POST	Import NDJSON

/api/saved_objects/_export	POST	Export NDJSON
/api/saved_objects/_resolve_import_errors	POST	Resolve import conflicts

+ 6 more internal endpoints

Track 2: Search/Query API → OpenSearch (12 Endpoints)

Endpoint	Method	Description
/internal/search/{strategy}/{id?}	POST	Search execution
/internal/search/{strategy}/{id}	DELETE	Cancel search
/internal/_msearch	POST	Multi-search
/api/console/proxy	POST	OpenSearch proxy (passthrough)
/api/opensearch-dashboards/suggestions/values/{index}	POST	Autocomplete

+ 7 more data/search endpoints

Other APIs: Case-by-Case Evaluation (~60 endpoints)

- Console (3): Already OpenSearch proxy - no change
- Data Importer (4): Uses OpenSearch - migrate
- Application Config (4): Uses OpenSearch - migrate
- Query Enhancements (7): Uses OpenSearch - migrate
- Workspace (8): Uses Saved Objects - depends on Track 1 success
- Others: TBD based on backend design decisions

Appendix B: Reference Materials

- **OpenAPI Spec Baseline:** mustang/openapi-spec.yaml
- **API Mapping:** mustang/OSD_TO_OPENSEARCH_API_MAPPING.pdf
- **OpenSearch API Assignment:** mustang/OpenSearch_API_Usage_TeamAssignment.xlsx
- **OSD API Assignment:** mustang/OSD_API_Mapping_TeamAssignment.xlsx
- **Integration Tests:** cypress/integration/ directory

Contact:

- UI Engineering Manager: [Your Name]
- Backend Engineering Lead: [TBD]
- QA Lead: [TBD]
- Infrastructure Lead: [TBD]

Document Status: Updated for Dual-Track Architecture (S3 + OpenSearch)

Next Review Date: [Fill in date for weekly sync]

Approval Required From: Backend Team Lead, QA Lead, Infrastructure Lead, Product Manager